

이동형

DevOps Engineer | 이력서 & 경력기술서

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

소개

안녕하세요, DevOps Engineer 이동형입니다.

인프라 가용성 99.9% 를 목표로, 모든 변경의 코드화(IaC)와 장애 대응의 체계적 문서화(포스트모템 자산화) 프로세스를 정착시켜 운영 효율성을 높여왔습니다. 단순한 운영을 넘어 각 업무의 의미와 목적을 고민하고 더 나은 방향을 찾는 습관이 가장 큰 강점입니다.

AWS·GCP 클라우드와 OpenStack 기반 온프레미스 PaaS 인프라 설계·구축, Kubernetes(EKS·GKE·온프레미스) 운영, Terraform·Ansible IaC, GitLab CI·GitHub Actions·ArgoCD·Jenkins 기반 CI/CD 파이프라인 구축에 능숙합니다. PLG·ELK Stack 으로 모니터링·로깅 시스템을 구축해왔고, Python·Bash 로 운영 자동화도 직접 다룹니다.

그리고 BigQuery·Dataflow·Cloud Functions·Cloud Scheduler 기반 데이터 파이프라인 자동화(멀티 소스 → BigQuery → Google Sheets, Terraform IaC)와 GPU 서버 위 Ollama + Open WebUI 사내 LLM 서비스 구축·운영 등 데이터 엔지니어링·AI/ML 인프라 영역으로 강점을 넓히고 있으며, git-bridge·Slack APK 배포 봇·static-file-server 같은 사내 업무 자동화·DevOps Tool 도 직접 개발하고 있습니다.

3개 회사에서 AWS → GCP 클라우드 마이그레이션, 하이브리드 아키텍처 설계, 온프레미스 PaaS 솔루션 구축을 단독·주도적으로 수행하며 AWS 비용 20% 절감(하이브리드 전환), GCP 마이그레이션 후 리소스 최적화로 비용 50% 절감, 배포 시간 50~67% 단축, 수동 작업 90% 이상 자동화의 정량적 성과를 달성했습니다.

앞으로도 인프라 아키텍처 고도화와 조직의 생산성 향상에 기여할 수 있는 엔지니어가 되고자 합니다.

강점

신속한 업무 파악 및 적응

- 새로운 환경·도구에 대한 빠른 적응력을 바탕으로 다양한 프로젝트에 즉시 기여

체계적인 문서화와 지식 공유

- 모든 작업 과정을 문서로 정리하고 팀과 공유하여 조직의 지식 자산 구축에 기여

기록 기반의 지속적 개선

- 장애 대응, 인프라 변경, 트러블슈팅 등 모든 작업을 기록 기반으로 관리하여 재발 방지 및 개선 추구

풀스택 인프라 경험

- 클라우드(AWS, GCP), 온프레미스, 네트워크, CI/CD, 모니터링을 아우르는 엔드투엔드 인프라 경험 보유

기술 스택

클라우드	AWS, GCP, OpenStack
컨테이너 오케스트레이션	Kubernetes (EKS/GKE 멀티 클러스터 운영, 온프레미스 HA 구성 및 CNI/CSI 최적화)
Infrastructure as Code	Terraform, Ansible, Vagrant, Packer
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, Jenkins
모니터링 & 로깅	PLG Stack, ELK Stack, EFK Stack, Thanos
자동화	Go, Python, Bash Scripting
데이터베이스	MySQL, PostgreSQL, Redis, MongoDB
OS & 서버	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS, GCP GKE 프로덕션 환경 운영 및 비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)
- Terraform을 활용한 IaC 기반 인프라 자동화 및 버전 관리 (GCP 리소스 100% 코드화)

- 온프레미스-클라우드 하이브리드 아키텍처 설계 및 마이그레이션 경험

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions, GitLab CI, ArgoCD를 활용한 CI/CD 파이프라인 설계 및 고도화
- 배포 시간 50~67% 단축 및 주간 배포 3배 증대로 Time to Market 단축, 롤백 시간 95% 감소 (30분 → 1~2분) 달성
- Jenkins 기반 Unity Android/iOS 빌드 자동화 및 Slack 연동 배포 알림 구축

모니터링 및 오피버빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- PLG Stack (Prometheus, Loki, Grafana) 기반 메트릭·로그 통합 모니터링 구축
- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- MTTR(평균 장애 복구 시간) 60~70% 개선, SLI/SLO 기반 서비스 가용성 99.9% 달성

자동화 및 효율화

Python, Bash를 활용한 운영 업무 자동화 및 개발 생산성 도구 구축

- 데이터 수집, 배포 프로세스, 문서 관리 자동화로 수동 작업 90% 이상 감소
- 내부 LLM 서비스(Ollama + Open WebUI) 구축으로 사내 보안 가이드라인 100% 준수 및 개발팀 생산성 향상
- GitLab Webhook 기반 문서 자동 동기화, Slack Bot 기반 APK 배포 알림 등 사내 도구 개발

Kubernetes 플랫폼 현대화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 11 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 11 Gateway CR 무중단 마이그레이션 (cutover 11/11, wildcard TLS 통합)
- ECK Operator 기반 로깅 스택을 로컬 차트 → OCI 차트로 무중단 전환 (CR/PVC 이름 보존, cluster_uuid 불변, 데이터 무손실)
- 오픈소스 Helm 차트 4종(nginx-gateway-cr, elasticsearch-eck, kibana-eck, certmanager-letsencrypt) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

데이터 엔지니어링 및 AI/ML 인프라

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버(RTX 3060) 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영, 외부 AI SaaS 구독 대체로 월 약 \$100 절감 및 사내 보안 가이드라인 100% 준수
- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0% 달성

주요 경력 상세 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 DevOps 인프라 전담 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. 개발 서버와 테스트 서버는 온프레미스에, QA와 Production 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중입니다.

현재 서비스 중인 게임 '소울즈'에 대해 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용 분석을 수행한 결과, 개발·테스트 환경은 트래픽 변동이 적어 클라우드의 탄력성이 불필요했고, QA·Production 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 개발·테스트 환경은 온프레미스로 전환하고, QA·Production 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다. Terraform과 Ansible IaC 통합으로 양쪽 환경의 운영 표준을 일원화(IaC화 100%)하여, 운영 복잡도 증가 없이 일관된 인프라 관리를 실현했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- Kubernetes IPVS 모드에서 ClusterIP 서비스 간 통신 시 간헐적 타임아웃 발생. Cilium eBPF 기반 패킷 흐름 분석 및 IPVS conntrack 임계치 튜닝으로 해결하고, Cilium Network Policy 기반 zero-trust 보안 모델을 구축하여 파드 간 트래픽 가시성 및 보안 강화
- Harbor (온프레미스) ↔ ECR (AWS) 간 이미지 동기화를 수동으로 처리하면서 cross-cluster 배포 시 외부 인터넷 경유 push/pull 이 빈번해져 빌드 시간·비용·실패율이 증가. Harbor native Replication (Harbor → ECR, event-based 정책) 으로 push 즉시 자동 복제 + 반대 방향은 ECR push 이벤트를 EventBridge → GitHub Actions workflow 로 트리거해 `skopeo copy` 로 Harbor 에 미러링, 양방향 동기화 자동화로 배포 시간 40% 단축
- 게임 서비스 rolling deploy 중 ALB target deregistration delay (기본 300초 → 60초 단축) 와 Pod SIGTERM 처리 시점 미스매치로 일부 세션 connection drop 발생. preStop hook 에 sleep 60s 추가 (readiness probe fail 유도로 ALB 가 unhealthy 인식 후 신규 요청 차단) + terminationGracePeriodSeconds 90초 (ALB drain 60s + 진행 중 요청 처리 30s) 조정 + 게임 서버 컨테이너에 SIGTERM 핸들러 구현으로 connection drop 0건 zero-downtime 배포 달성

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별 역할 분리로 운영 안정성 확보 및 장애 전파 방지

기술 스택

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. Kaniko를 활용한 Docker 이미지 빌드로 Docker-in-Docker 의존성을 제거하고, 멀티 환경(alpha/review/dev/staging) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- ArgoCD Application 의 syncPolicy.automated.selfHeal=true 가 운영자의 임시 hot-fix 를 즉시 되돌려 장애 디버깅이 어려워지는 문제 발생. prod 는 selfHeal=false + manual sync window 로 분리하고 dev/staging 만 selfHeal=true 유지, 모든 sync 이벤트 Slack 알림 연동으로 변경 추적성과 안정성 동시 확보
- GitLab 업그레이드 후 GPG 키 만료로 CI Runner에서 패키지 설치 실패. GPG 키 갱신 프로세스를 자동화하여 재발 방지
- Kaniko 빌드 시 캐시 무효화로 빌드 시간 급증. --cache=true --cache-ttl=24h --snapshot-mode=redo 옵션 조합으로 해결

성과

- 배포 시간 67% 단축 (30분 → 10분), 수동 작업 자동화 90% 달성
- 배포 자동화로 주간 배포 횟수를 3배 이상 증대 (1~2회 → 5~7회)시켜 신규 피처의 시장 출시 주기(Time to Market)를 단축하고, 배포 리소스 효율화로 개발팀이 본연의 기능 개발에 집중할 수 있는 환경 조성

기술 스택

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

프로젝트 3: 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화)

기여도 100% (시스템 설계 및 구축) 2025.06 ~ 현재 (운영 및 고도화)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었습니다.

트러블슈팅

- 게임 서버 로그의 null 바이트(\x00)로 Logstash 파싱 실패. mutate 필터 gsub으로 사전 제거하여 해결
- Elasticsearch 필드 매핑 충돌(mapper_parsing_exception). 인덱스 템플릿 명시적 매핑 및 타입 변환 필터 적용으로 해결
- 디스크 워터마크 초과로 인덱스 read-only 전환. ILM 정책으로 7일 초과 인덱스 자동 삭제하여 안정화
- [EFK] Fluent Bit NFS WAL 쓰기 실패. 커스텀 PVC 템플릿 패치 + 업그레이드 스크립트 자동 보존으로 해결
- [EFK] Fluentd buffer overflow 로그 유실. chunk_limit_size/flush_interval 튜닝으로 해결
- [ECK] Kibana 9 secure-cookie 강제로 HTTP 환경 로그인 실패. tls.disabled + publicBaseUrl http:// + secureCookies:false 조합으로 우회
- [ECK] 3.3.x stackconfigpolicy-controller의 cache-scope mismatch로 17분 간격 reconcile error 반복. managedNamespaces:[] (cluster-wide watch)로 우회, RBAC는 이미 cluster-scoped라 권한 델타 없음
- [ECK] Fluentd ES9 variant 부재 → fluent-plugin-elasticsearch 5.4.4의 suppress_type_name:true로 ES8 variant 이미지와 ES 9.0 호환성 확보, active marker 엔드투엔드 검증

접근 전략

Phase 1 (ELK 구축): Elasticsearch + Logstash + Kibana + Filebeat 로 중앙 집중식 로깅 시스템 구축. 노드/컨테이너 로그를 Filebeat 로 수집 → Logstash 가 JSON 파싱·필드 매핑 → Elasticsearch 저장, Kibana 대시보드로 실시간 모니터링·검색 제공.

Phase 2 (EFK 전환): Logstash JVM 메모리 부담(1GB+) 과 Filebeat 의 Helm 생태계 불일치를 해소하기 위해 Fluent Bit + Fluentd 기반 EFK Stack 으로 전환. C 기반 경량 수집기 Fluent Bit DaemonSet 이 K8s 메타데이터 자동 태깅과 함께 수집, Fluentd 가 멀티 output 가공/필터링 후 Elasticsearch 전달. 차트 업그레이드 자동화 스크립트로 upstream 차트 자동 업그레이드 시 커스텀 PVC 템플릿을 자동 보존.

Phase 3 (ECK Operator + Stack 9.0): Elastic 공식 Helm Chart 업데이트 중단과 CRD 기반 선언형 관리 주류화에 따라, Stack 8.5.1 → 9.0.0 major upgrade와 함께 ECK Operator 3.3.2를 elastic-system ns에 도입. ES/Kibana를 Custom Resource로 재정의(CR wrapper 로컬 차트)하고 monitoring → logging 병렬 배포 후 cutover로 무중단 전환. TLS 인증서 자동 회전(1년 / 30일), elastic 계정 패스워드 Helm template 관리, CR spec.version 한 줄 변경만으로 롤링 업그레이드 가능한 체계 확립.

성과

- 로그 검색 시간 90% 단축, MTTR 70% 개선으로 서비스 가용성 99.9% 상시 유지
- EFK 전환으로 로그 수집기 메모리 70% 감소 (Logstash 1GB → Fluent Bit 50MB)
- 일 평균 1GB 로그 실시간 처리, Helm 기반 관리 통일 및 업그레이드 자동화
- ECK 전환으로 TLS·패스워드·StatefulSet 업그레이드의 수동 운영 부담 제거, Stack 9.0 major upgrade로 2년간 누적된 업데이트 격차 해소
- Operator secureMode + ServiceMonitor로 controller-runtime 메트릭을 kube-prometheus-stack에 자동 등록

기술 스택

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% 2025.02 ~ 현재

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins 빌드 완료 시 Slack에 QR 코드와 함께 자동 알림. QA팀 전달 시간 90% 단축

기술 스택

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.08)

GitLab Webhook + Google Drive API로 Push 시 자동 동기화. 문서 관리 시간 80% 감소

기술 스택

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. 내부 LLM 서비스 구축 (4주, 2026.01)

온프레미스 GPU 서버(RTX 3060)에 Ollama + Open WebUI를 Kubernetes에 배포. 개발팀 15명 · 일 평균 100건 쿼리 처리, 외부 AI SaaS 구독 대체로 월 약 \$100 절감. 사내 보안 가이드라인 100% 준수 및 외부 데이터 유출 리스크 제거

기술 스택

Ollama, Open WebUI, Kubernetes, NFS, GPU (RTX 3060)

4-4. Git 저장소 미러링 도구 개발 (4주, 2026.02)

Go 기반 양방향 Git 미러링 도구(git-bridge) 개발. 유닛 테스트 및 GitHub Actions CI/CD 파이프라인으로 코드 품질 확보. CodeCommit/GitLab/GitHub 간 자동 동기화, 저장소 10개 · 일 평균 30건 동기화 100% 자동화

기술 스택

Go, AWS SQS, GitLab Webhook, GitHub Webhook, Kubernetes, Docker

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

Go 기반 경량 정적 파일 서버(static-file-server)를 직접 개발하고 GitHub Pages에 Helm repository를 운영. 다크모드 디렉토리 리스팅 UI, 파일 프리뷰 (이미지/비디오/PDF/텍스트), 검색·URL 해시 공유, 다중 선택 + ZIP 일괄 다운로드, Gzip, Prometheus 메트릭, JSON 로깅, APK Content-Type 자동 설정 기능 제공.

자체 Helm chart로 Kubernetes에 배포(NFS nfs-client-nopath, Retain, 5Gi)하고 기존 halverneus/static-file-server:v1.8.11을 _deprecated로 완전 대체. 일 평균 30건 다운로드 · 주 5회(매일 빌드) APK 배포 트래픽 처리. Apache-2.0 오픈소스 공개.

기술 스택

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% 2025.12 ~ 현재

클러스터 가시성, 운영 안정성, 네트워크 현대화를 목표로 모니터링 체계를 고도화하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다. NGF 마이그레이션 과정에서 도출된 CR 관리 구조와 ECK Operator 기반 로깅 스택을 각각 오픈소스 Helm 차트로 공개했습니다.

5-1. 모니터링 & 알림 고도화 (2026.01 ~ 현재)

kube-prometheus-stack 기반으로 18개 커스텀 알림 규칙(Node 9개, Pod 2개, Cilium 4개)과 10개 커스텀 Grafana 대시보드를 설계하고, MySQL·Redis·Elasticsearch·PostgreSQL exporter를 통합했습니다. 물리서버 4대와 VM에 Ansible로 node-exporter를 자동 배포하고, Cilium CNI agent/operator 메트릭을 kubernetes_sd_configs로 수집.

Alertmanager에서 severity 기반 Slack 라우팅과 inhibit rules를 구성해 알림 품질을 확보했습니다.

기술 스택

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 2026.04)

Kubespray 업그레이드 자동화 스크립트로 사전 검증·백업·호환성 확인, 9단계 헬스체크 스크립트로 검증 시간 90% 단축.

18개 Helm 차트 업그레이드 프레임워크 + 4가지 캐노니컬 템플릿 동기화 도구(check로 CI drift 감지, apply로 일괄 전파)로 스크립트 본문 일관성 자동 유지.

K8s 인증서 자동 갱신 스크립트 + crontab(노드 10대, 6개월마다 새벽 3시) 주기 실행으로 인증서 만료로 인한 API Server 장애 사전 방지

기술 스택

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

Ingress-nginx 컨트롤러 11개 인스턴스(기본 + public-a~j, MetalLB LoadBalancer IP 11개 고정)를 공식 후계자 NGF 2.x 기반 단일 컨트롤플레인 + 11 Gateway CR 구조로 전환. MetalLB IP를 그대로 유지하여 DNS/방화벽 무변경 병렬 배포 + 점진 cutover 방식 채택(클래스당 실 다운타임 30~60초). Phase 0~7+ 7단계(MetalLB 풀 확장 → NGF 설치 → 관측 스택 5종 → HTTP-only/ApplicationSet 9개 → HTTPS 강제 앱(Harbor·Vaultwarden) → IP swap cutover 11/11 → Ingress 정리)로 진행. 어노테이션 10종을 Gateway API + NGF CRD(HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy)로 1:1 매핑, HTTPRoute 차트 values 스키마를 4개 차트에 통일해 ApplicationSet 9개 일괄 전환에 재사용.

self-signed wildcard cert 1장(*.concrit.us, 10년)으로 애플 TLS Secret 2장을 통합해 수동 갱신 부담 50% 감소.

기술 스택

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. nginx-gateway-cr 오픈소스 Helm 차트 공개 (2026.04)

NGF 마이그레이션 과정에서 추출한 CR 구조를 다듬어, NGF upstream 차트가 제공하지 않는 테넌트 레벨 리소스

(Gateway·NginxProxy·ReferenceGrant·ServiceMonitor·PodMonitor)를 배포하는 범용 Helm 차트로 오픈소스 공개. multi-Gateway 친화적 스키마와 NGF 2.x 권장 PodMonitor 템플릿 포함, OCI + 전통 Helm 레포 양쪽 배포 (Apache-2.0, ArtifactHub)

기술 스택

Helm Chart, Gateway API, NGINX Gateway Fabric, Prometheus Operator, OCI Registry, ArtifactHub

5-5. ECK Operator 기반 Elasticsearch / Kibana Helm 차트 공개 (2026.04)

charts/elasticsearch-eck + charts/kibana-eck 두 차트를 설계해 ECK Operator 기반 CR·Secret·HTTPRoute·Ingress·ServiceMonitor·NetworkPolicy 를 단일 values 로 배포하도록 구성하고, values.schema.json(draft-07) 입력 검증·Minimal/HA 프리셋·환경별 storageClass 매핑을 README 에 명시.

OCI(ghcr.io/somaz94/charts)-gh-pages Helm 레포·ArtifactHub monitoring-logging 카테고리(Apache-2.0, 0.1.1) 세 경로로 동시 배포.

mgmt 클러스터 무중단 전환 시 CR·PVC·Secret 이름 보존으로 cluster_uuid 불변, ES pod 재시작 0회·Kibana rolling 1회로 데이터 무손실 완료. HA 롤링 kind 검증 3/3 PASS, 실 클러스터 설치 Elasticsearch 71초·Kibana 57초 Ready. scripts/upgrade-sync/templates/oci-chart-version.sh canonical 신규 작성으로 사내 OCI 차트 버전 추적 절차도 함께 정비.

기술 스택

Helm Chart, ECK Operator, Elasticsearch, Kibana, Gateway API, OCI Registry, ArtifactHub, Prometheus Operator, Kubernetes

프로젝트 6: 인프라 보안 체계 구축

기여도 100% 2026.04 ~ 현재

6-1. Vaultwarden 비밀번호 관리 시스템 (2026.04)

Vaultwarden을 K8s에 Helm 배포, GitLab SSO 연동, 조직/컬렉션 기반 접근 제어. 자동 백업(날짜별, 30일 retention) + 대화형 restore. 사용자 70명 · 시크릿 50건 중앙 관리 체계로 전환

기술 스택

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임 및 블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로의 대규모 클라우드 마이그레이션 프로젝트를 총괄했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 재구축

기여도: 인프라 설계·마이그레이션 총괄 70%, CI/CD 파이프라인 100% 7개월 (2023.05 ~ 2023.12)

문제

AWS 비용 지속 상승(월 \$10,000+, NAT Gateway·데이터 전송 비용 高), 멀티 리전 게임 서비스를 위한 GCP 글로벌 LB 필요, Jenkins 기반 배포 시스템의 설정 복잡성 및 배포 이력 추적 불가

접근 전략

Shared VPC (Host / Service Project 분리) 와 Workload Identity Federation 을 도입해 GitHub Actions 가 서비스 계정 키 없이 임시 토큰으로 GCP 리소스에 인증하도록 구성. DNS 는 서브도메인 위임 (Hosting.kr → AWS Route53 → GCP Cloud DNS) 으로 사전 검증 후, 최종 NS 변경 시점에만 점검 공고로 무중단 cutover.

Filestore 의 SSD 최소 2.5TB 비용 문제는 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 회피. Cloud Armor 로 region 차단 + IP 화이트리스트 WAF 정책을 구성하고, Cloud CDN 의 URL Map 을 HTTP / HTTPS 두 개로 분리하여 HTTP → HTTPS 강제 리다이렉트와 정적 자산 캐싱 동시 활성화.

트러블슈팅

- AWS ALB → GCP 글로벌 LB 전환 시 헬스체크/백엔드 구성 차이로 라우팅 이슈. GCP NEG 구조 분석 후 재구성
- RDS → Cloud SQL 마이그레이션 시 캐릭터셋 차이로 데이터 깨짐. 사전 검증 스크립트 및 단계별 정합성 체크로 해결
- AWS/GCP 서브넷 모델 차이(AZ vs 리전)로 네트워크 설계 이슈. CIDR 재설계 및 GCP 태그 기반 방화벽 전환
- GKE Ingress + BackendConfig 의 healthcheck requestPath 응답 누락으로 503 발생. 해당 경로 응답 보장 + ManagedCertificate / FrontendConfig 정렬로 해결
- CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드 실패. URL Map 을 HTTP / HTTPS 두 개로 분리 (default_url_redirect.https_redirect=true) 후 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 배포 시간 50% 단축, 롤백 95% 감소
- 전체 GCP 인프라 Terraform IaC화 (TFLint/Checkov 정적 분석, Terratest 단위 테스트 도입), Git 커밋 기반 배포 이력 100% 추적
- 리소스 복사 방식으로 다운타임 최소화, 최종 DNS 전환은 게임 서비스 특성을 반영해 약 2시간 내외 점검 공지 후 전환 수행
- Workload Identity Federation 도입으로 GitHub Actions 의 GCP 서비스 계정 키 관리 부담 제거 및 키 유출 리스크 원천 차단

기술 스택

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하여 분석가가 매일 2~3시간 소요, 휴먼 에러로 인한 데이터 누락 빈번 발생

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery 로 적재 (MongoDB 는 Dataflow ETL, CloudSQL 은 직접 connection, GA/Dune 은 API) 후, Cloud Functions + Cloud Scheduler 가 Daily / Monthly 단위로 자동 트리거하고 Google Sheets API 로 분석가용 사이트에 자동 적재. BigQuery 단의 중복 데이터는 별도 Cloud Function 으로 정기 제거.

전체 인프라 (Dataset · Cloud Function · Scheduler · Storage · Artifact Registry) 는 Terraform IaC 로 관리.

성과

- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0%

기술 스택

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Docker

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% 1개월 (2024.04)

문제

Kubernetes 클러스터·애플리케이션 실시간 모니터링 체계가 없어 사후 대응만 가능했고, 장애 원인 파악에 과도한 시간이 소요됨

성과

- SLI/SLO 수립 및 Alertmanager 알람 노이즈 최적화, 장애 사전 감지율 80%, 장애 대응 시간 60% 감소, 서비스 가용성 99.5% → 99.9%

기술 스택

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했고, 금융권 및 공공기관 대상 온프레미스 클라우드 인프라 설계·구축 및 운영 교육을 수행했습니다.

주요 프로젝트: PaaS 솔루션 구축 · DP사 교육 · 인증서 자동화 · Ansible 구성 관리

성과

- 5개 고객사에 PaaS 솔루션 구축, Kubernetes HA 99.9% 가용성
- DP사 운영팀 Kubernetes 자체 운영 체계 확립 (6개월, 수강자 평균 10명 · 회당 4시간)
- 인증서 갱신 주기 10배 연장 (1년 → 10년), 연간 50시간 운영 부담 절감
- Ansible 기반 서버 프로비저닝 및 구성 관리 자동화로 환경 불일치(Configuration Drift) 이슈 제로화

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, kubeadm, Rocky Linux/CentOS

이테크시스템

2022.01 ~ 2022.04

System Engineer

Hardware Server 및 SAN 스토리지 구축, DevOps 분야로의 커리어 전환을 위해 이직.

오픈소스 프로젝트

Kubernetes CRD

[k8s-namespace-sync](#) — Kubernetes 네임스페이스 리소스를 클러스터 간 자동 동기화

[helios-lb](#) — Kubernetes용 커스텀 로드밸런서 컨트롤러

[network-policy-generator](#) — Kubernetes NetworkPolicy 자동 생성 컨트롤러

GitHub Actions

[Compress Decompress Action](#) — 워크플로우 내 파일 압축/해제

[Image Tag Updater](#) — YAML/Helm 파일 이미지 태그 자동 업데이트

[Go Git Commit Action](#) — Go 기반 Git 커밋 생성 및 푸시

[Extract Commit Action](#) — Git 커밋 SHA, 메시지, 작성자 정보 추출

[Multi Git Mirror Action](#) — 여러 Git 저장소 간 미러링 자동화

[Kube Diff Action](#) — Kubernetes 리소스 변경사항 비교 및 리포트

Ansible Galaxy

[Ansible K8s IAC Tool](#) — K8s 및 IaC 도구 자동 설치 컬렉션

[Ansible User Management](#) — Linux 사용자 및 SSH 키 관리 역할

DevOps Tools

- [bash-pilot](#) — AI 기반 Bash 명령어 어시스턴트 CLI 도구
- [git-bridge](#) — Git 저장소 간 미러링 및 동기화 CLI 도구
- [static-file-server](#) — 디렉토리 UI·파일 프리뷰·검색·ZIP 일괄 다운로드 지원 Go 기반 정적 파일 서버

Chrome Extensions

- [Dev Toolkit](#) — 개발자 유틸리티 모음
- [QRify](#) — URL/텍스트 QR 코드 즉시 생성

학력

충남대학교 행정학부 (학사편입)	2018.03 ~ 2020.08
국가평생교육진흥원 경영학사 (학점은행제)	2016.09 ~ 2018.02

자격증

정보처리기사	2021.11
AWS Certified Solution Architect - Associate (만료)	2021.11
네트워크 관리사 2급	2021.10
리눅스 마스터 2급	2021.10
컴퓨터활용능력 1급	2021.04
TOEIC Speaking Test - 130점/Intermediate Mid 3 (만료)	2021.02

스터디

T101(Terraform) 스터디	2023.08 ~ 2023.10
AEWS(EKS) 스터디	2023.04 ~ 2023.06
PKOS(쿠버네티스) 스터디	2023.01 ~ 2023.02
Cloud Security 교육	2022.02 ~ 2022.04
보안솔루션 운영 전문가 양성과정 (국비교육)	2021.07 ~ 2021.12